H7137 — Numerical Modelling & Engineering Simulations Portfolio

Candidate Number: **295536**

University of Sussex

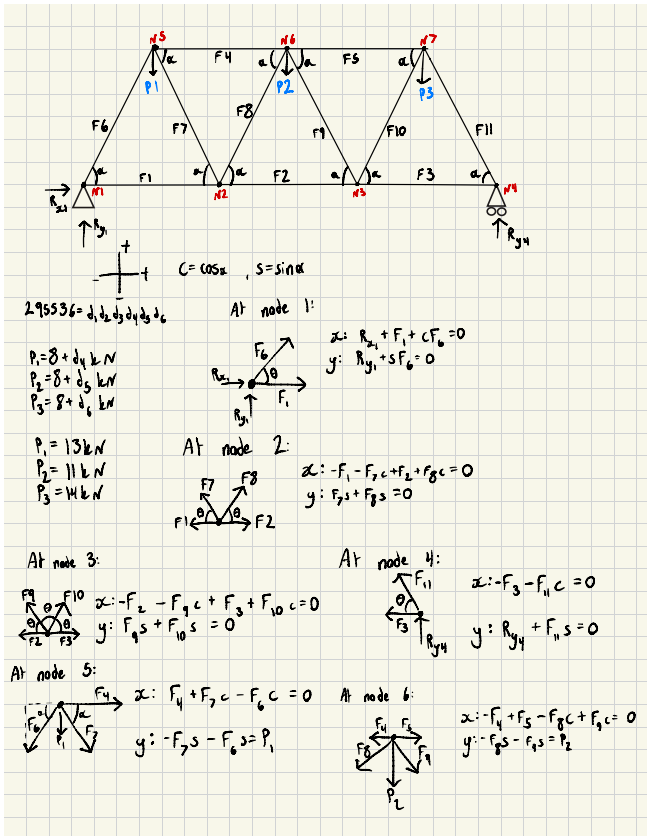
2025–2026

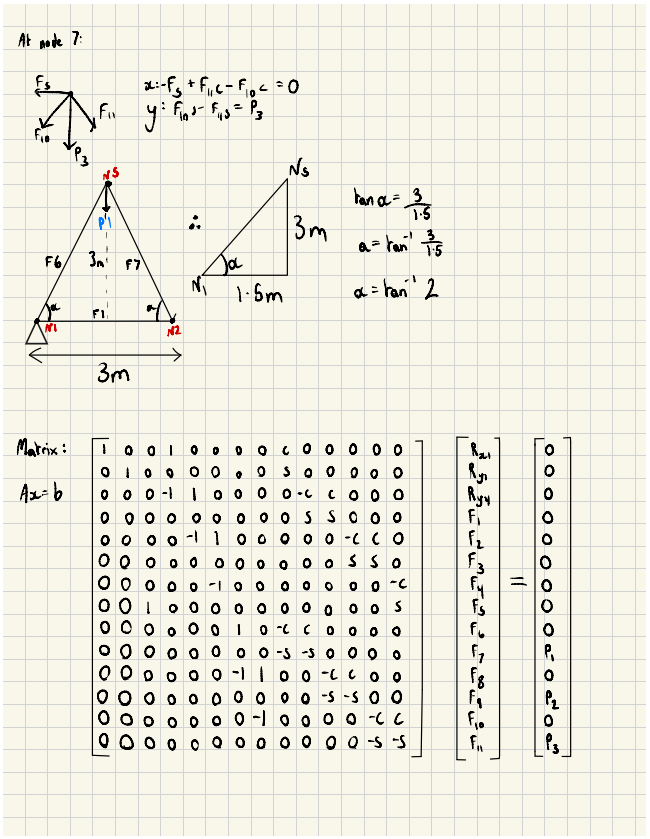
Contents

Problem 1 — Static Truss Solver

Part A: Derivation and Matrix System

Hand calculations (scanned):





For candidate 295536, $d_4 = 5$, $d_5 = 3$, $d_6 = 6$, giving $P_1 = 13\,\text{kN}$, $P_2 = 11\,\text{kN}$, $P_3 = 14\,\text{kN}$. Diagonal angle $\alpha = \arctan(3/1.5)$ yields $c = \cos\alpha$, $s = \sin\alpha$. Unknowns are ordered as:

$$\mathbf{x} = \left[R_{x1},\ R_{y1},\ R_{y4},\ F_1,\ F_2,\ \ldots,\ F_{11}\right]^\top$$

Applying $\sum F_x = 0$ and $\sum F_y = 0$ at each node in sequence N1–N7 gives the 14×14 system $\mathbf{Ax} = \mathbf{b}$ [1]:

$$\mathbf{A} = \begin{bmatrix}
1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & c & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & s & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 & 0 & -c & c & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & s & s & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -1 & 1 & 0 & 0 & 0 & 0 & 0 & -c & c & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & s & s & 0 \\
0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -c \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & s \\
0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & -c & c & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -s & -s & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 1 & 0 & 0 & -c & c & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -s & -s & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & -c & c \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -s & -s & -s
\end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 13 \\ 0 \\ 11 \\ 0 \\ 14 \end{bmatrix} \text{ kN}$$

Part B: Command Line Output and Global Checks

	x	Loading
Rx1	0	{'Reaction'}
Ry1	18.667	{'Reaction'}
Ry4	19.333	{'Reaction'}
F1	9.3333	{'Tension'}
F2	15	{'Tension'}
F3	9.6667	{'Tension'}
F4	-12.167	{'Compression'}
F5	-12.333	{'Compression'}
F6	-20.87	{'Compression'}
F7	6.3355	{'Tension'}
F8	-6.3355	{'Compression'}
F9	-5.9628	{'Compression'}
F10	5.9628	{'Tension'}
F11	-21.615	{'Compression'}

```
Most loaded member:  F11 (-21.62 kN, Compression)
```

Global checks:

- $\sum F_y = 0$: $R_{y1} + R_{y4} = P_1 + P_2 + P_3 = 38\,\text{kN}$ ✓
- $\sum F_x = 0$: $R_{x1} \approx 0$ ✓
- Moment about N1: $R_{y4} \times 9 - (13 \times 1.5 + 11 \times 4.5 + 14 \times 7.5) = 0$ ✓

Part C: Engineering Interpretation

Member 11 carries the largest force at $-21.62\,\text{kN}$ in compression, while F2 carries the largest tension at $+15.00\,\text{kN}$, member 11 governs because it has to resolve the full shear in the bay next to the roller support, where the asymmetric loading concentrates toward N7.

MATLAB Code

Listing 1: Problem1.m — Static Warren Truss Solver

```
clc; clear;
% Initialising loads according to my candidate number: 295536
% All loads are in kN
P1=8+5; % Load at node 5
P2=8+3; % Load at node 6
P3=8+6; % Load at node 7
% Initialising angle
alpha=atan(3/1.5);
c=cos(alpha);
s=sin(alpha);

P=P1+P2+P3;

% Defining matrices A and b
% A = Matrix of unknown variables
A = [1 0 0 1 0 0 0 0 c 0 0 0 0 0;
     0 1 0 0 0 0 0 0 s 0 0 0 0 0;
     0 0 0 -1 1 0 0 0 0 -c c 0 0 0;
     0 0 0 0 0 0 0 0 0 0 s s 0 0 0;
     0 0 0 0 -1 1 0 0 0 0 0 -c c 0;
     0 0 0 0 0 0 0 0 0 0 0 0 s s 0;
     0 0 0 0 0 -1 0 0 0 0 0 0 0 -c;
     0 0 1 0 0 0 0 0 0 0 0 0 0 s;
     0 0 0 0 0 0 0 1 0 -c c 0 0 0 0;
     0 0 0 0 0 0 0 0 0 -s -s 0 0 0 0;
     0 0 0 0 0 0 -1 1 0 0 -c c 0 0;
     0 0 0 0 0 0 0 0 0 0 -s -s 0 0;
     0 0 0 0 0 0 0 0 -1 0 0 0 0 -c c;
     0 0 0 0 0 0 0 0 0 0 0 0 0 -s -s;];

% b = Matrix of known variables
b = [0;0;0;0;0;0;0;0;0;P1;0;P2;0;P3];

% Solving for the forces
x = A\b;
% Assigning values from matrix x to the reaction forces
Rx1 = x(1);
Ry1 = x(2);
Ry4 = x(3);

```

```
% Global checks to ensure the force calculations are correct
if abs(P - Ry1 - Ry4) < 1e-6 && abs(Rx1) < 1e-6 && abs(Ry4*9 - (P1*1.5 + P2*4.5 + P3*7.5)) < 1e-6

    % Creating the parameters for the table
    Loading = cell(14,1);
    % Identifying forces as tension or compression
    % Positive is tension, negative is compression
    for i = 1:14
        solution = x(i);
        if i <= 3
            Loading{i} = 'Reaction';
        elseif solution > 1e-4
            Loading{i} = 'Tension';
        elseif solution < -1e-4
            Loading{i} = 'Compression';
        else
            Loading{i} = '0';
        end
    end

    % Creating a table to display all the member forces
    beams = {'Rx1','Ry1','Ry4','F1','F2','F3','F4','F5','F6','F7','F8','F9','F10','F11'};
    ResultsTable = table(x, Loading, 'RowNames', beams);
    disp(ResultsTable);

    % Identifying the most highly loaded member
    TrussForces = x(4:end);
    [~, idx] = max(abs(TrussForces));
    fprintf('Most loaded member: F%d (%.2f kN, %s)\n', idx, TrussForces(idx), Loading{idx+3});

else
    % Displaying a message in case one of the global checks failed
    fprintf('An error has occurred. Equilibrium checks failed.\n')
end
```

Problem 2 — Static Truss Parametric Design Tool

Part A: Figures and Results

For candidate 295536, $P_2 = 11\,\text{kN}$ is fixed while $P_1 \in [0, 39]\,\text{kN}$ and $P_3 \in [0, 42]\,\text{kN}$ each span 100 linearly spaced values, producing 10 000 load combinations.

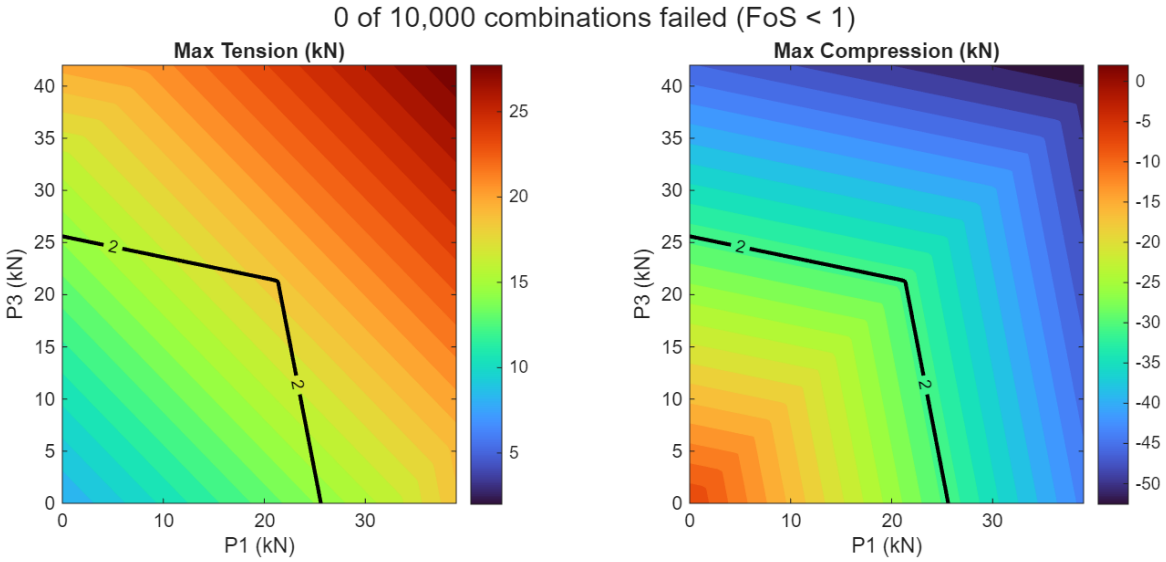


Figure 1: Maximum tension (left) and maximum compression (right). FoS = 2 solid black contour; FoS = 1 dashed black contour. *Note: No dashed line can be seen as no permutation achieved a FoS < 1.*

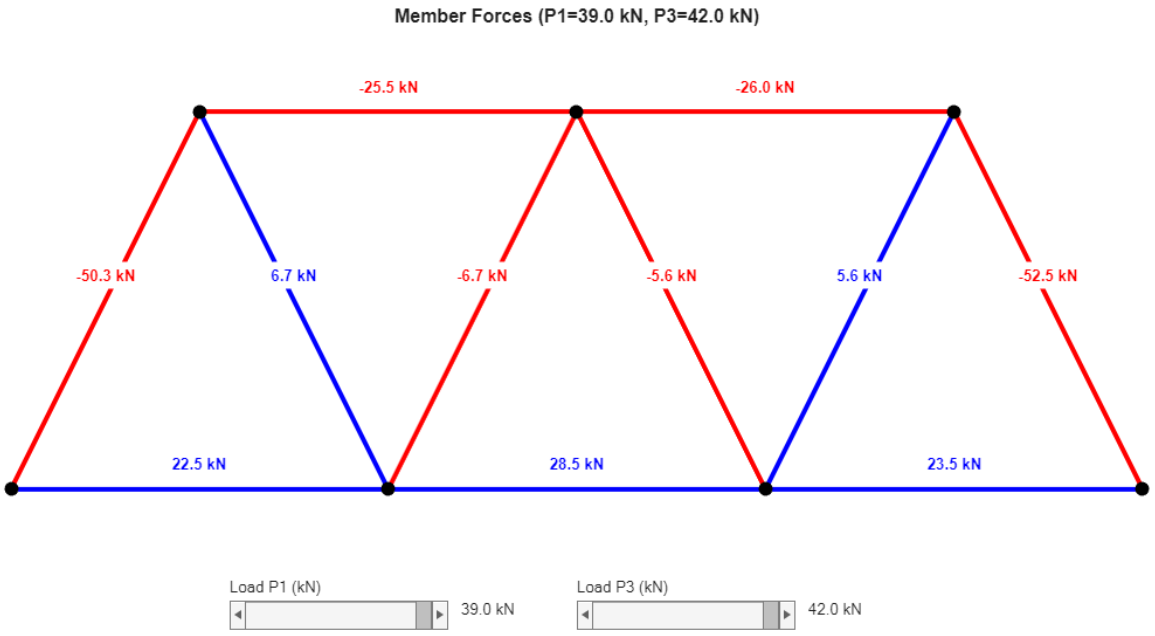


Figure 2: Interactive truss tool. Sliders adjust P_1 and P_3 in real time, member colours and force annotations update instantly. Blue = tension, red = compression.

Part B: Command Line Output

Member forces are tested against allowable limits of 40 kN in tension and 60 kN in compression.

$$\text{FoS} = \frac{F_{\text{allow}}}{|F_{\text{actual}}|} \tag{1}$$

```
Min FoS: 1.142

0 of 10,000 combinations failed (FoS < 1)
```

Part C: Engineering Interpretation

The truss is safe, zero failures across all 10 000 combinations, but the minimum FoS of 1.142, at the $(P_1, P_3) = (39, 42)$ kN corner, governed by member 11 in compression, falls below typical structural design margins of 1.5–2.0 [1], so service loading should be capped well inside the FoS = 2 contour shown in Figure 1.

MATLAB Code

Listing 2: solveproblem2.m — Truss Solver Function (P1 and P3 variable)

```
function x = solveproblem2(P1, P3)
    % Function accepting P1 and P3, outputting all forces.

    P2 = 8+3; % Based on candidate number
    alpha = atan(3/1.5);
    c = cos(alpha);
    s = sin(alpha);

    % A = Matrix of unknown variables
    A = [1 0 0 1 0 0 0 0 c 0 0 0 0 0;
         0 1 0 0 0 0 0 0 s 0 0 0 0 0;
         0 0 0 -1 1 0 0 0 0 -c c 0 0 0;
         0 0 0 0 0 0 0 0 0 s s 0 0 0;
         0 0 0 0 -1 1 0 0 0 0 0 -c c 0;
         0 0 0 0 0 0 0 0 0 0 0 s s 0;
         0 0 0 0 0 -1 0 0 0 0 0 0 0 -c;
         0 0 1 0 0 0 0 0 0 0 0 0 0 s;
         0 0 0 0 0 0 1 0 -c c 0 0 0 0;
         0 0 0 0 0 0 0 0 -s -s 0 0 0 0;
         0 0 0 0 0 0 -1 1 0 0 -c c 0 0;
         0 0 0 0 0 0 0 0 0 0 -s -s 0 0;
         0 0 0 0 0 0 0 -1 0 0 0 0 -c c;
         0 0 0 0 0 0 0 0 0 0 0 0 -s -s];

    % b = Matrix of known variables
    b = [0;0;0;0;0;0;0;0;0;P1;0;P2;0;P3];

    % Solve for the forces
    x = A\b;
end
```

Listing 3: Problem2.m — Design Space Sweep and Interactive Visualisation

```
clc; clear; close all;

% Calculate max loads based on my candidate number: 295536
% Loads vary from 0 up to 200% of the initial values
P1_max = (8+5) + 2*(8+5);
P3_max = (8+6) + 2*(8+6);

% Creating arrays of 100 load values each using linspace
P1_loads = linspace(0, P1_max, 100);
P3_loads = linspace(0, P3_max, 100);

% Preallocate arrays to store results for 10,000 load combinations
max_tension_results     = zeros(100, 100);
max_compression_results = zeros(100, 100);
FoS_results             = zeros(100, 100);
all_forces              = zeros(100, 100, 14); % Stores full solution vector for every case

% Limits for Factor of Safety check
tension_limit     =  40; % Max allowable tension in kN
compression_limit = -60; % Max allowable compression in kN

% Nested loops to cycle through all combinations of P1 and P3 loads
for i = 1:100
    for j = 1:100

        % Calls the function for the current load pair
        x = solveproblem2(P1_loads(i), P3_loads(j));

        % Save the full results
        all_forces(i, j, :) = x;

        % Extract only the 11 member forces (ignoring the 3 reaction forces)
```

```
        Forces = x(4:14);

        % Identify the highest tension and highest compression in the truss
        max_tension = max(Forces);
        max_compression = min(Forces);

        % Ensure tension is positive and compression is negative
        if max_tension < 0
            max_tension = 0;
        end

        if max_compression > 0
            max_compression = 0;
        end

        % Save the peak forces for the specific load
        max_tension_results(i,j)     = max_tension;
        max_compression_results(i,j) = max_compression;

        % Calculate Factor of Safety (Limit / Actual Force)
        FoS_t = inf; % Set to infinity if no tension exists
        FoS_c = inf; % Set to infinity if no compression exists

        if max_tension > 0
            FoS_t = tension_limit / max_tension;
        end
        if max_compression < 0
            FoS_c = compression_limit / max_compression;
        end

        % Store the lowest FoS for this load combination
        FoS_results(i,j) = min(FoS_t, FoS_c);
    end
end

% Results and failure counts
failed_perms = sum(FoS_results(:) < 1);
finite_FoS   = FoS_results(isfinite(FoS_results(:))); % Filter out infinity cases for the check
fprintf('Min FoS: %.3f\n', min(finite_FoS));
fprintf('%d of 10,000 combinations failed (FoS < 1)\n', failed_perms);

% Prepare grid for generating the contour plots
[P1_grid, P3_grid] = meshgrid(P1_loads, P3_loads);

% Create two side-by-side contour plots for tension and compression
figure('Position', [100, 100, 1000, 400]);
sgtitle(sprintf(' %d of 10,000 combinations failed (FoS < 1)', failed_perms));

% Subplot 1: Maximum Tension
subplot(1,2,1);
contourf(P1_grid, P3_grid, max_tension_results', 25, 'LineStyle', 'none');
colormap(gca, 'turbo');
colorbar;
xlabel('P1 (kN)'); ylabel('P3 (kN)');
title('Max Tension (kN)');
hold on;
% Draw contour lines specifically for FoS = 1 and FoS = 2
[C2,h2] = contour(P1_grid, P3_grid, FoS_results', [2 2], 'k',  'LineWidth', 2);
clabel(C2, h2, 'Color', 'k', 'FontSize', 9);
[C1,h1] = contour(P1_grid, P3_grid, FoS_results', [1 1], 'k--', 'LineWidth', 2);
clabel(C1, h1, 'Color', 'k', 'FontSize', 9);
hold off;

% Subplot 2: Maximum Compression
subplot(1,2,2);
contourf(P1_grid, P3_grid, max_compression_results', 25, 'LineStyle', 'none');
colormap(gca, 'turbo');
colorbar;
xlabel('P1 (kN)'); ylabel('P3 (kN)');
title('Max Compression (kN)');
hold on;
% Draw contour lines for FoS = 1 and FoS = 2
[C2,h2] = contour(P1_grid, P3_grid, FoS_results', [2 2], 'k',  'LineWidth', 2);
clabel(C2, h2, 'Color', 'k', 'FontSize', 9);
[C1,h1] = contour(P1_grid, P3_grid, FoS_results', [1 1], 'k--', 'LineWidth', 2);
clabel(C1, h1, 'Color', 'k', 'FontSize', 9);
hold off;

% Setup for customisable truss
% Define how nodes are connected and set the node coordinates
Member_links = [1,2; 2,3; 3,4; 5,6; 6,7; 1,5; 2,5; 2,6; 3,6; 3,7; 4,7];
Y = [0, 0, 0, 0, 3, 3, 3];
X = [0, 3, 6, 9, 1.5, 4.5, 7.5];

% Create the figure window
figure2 = figure('Name','Customisable Truss','Color','white','Position',[80 80 1100 640]);

% Setup the main area
truss_axes = axes('Parent',figure2,'Position',[0.07 0.20 0.88 0.72]);
axis(truss_axes, 'equal');
axis(truss_axes, 'off');
```

```
124
125 % Create text labels for the load sliders
126 createLabel(figure2, [0.24 0.16 0.13 0.03], 'Load P1 (kN)');
127 createLabel(figure2, [0.51 0.16 0.15 0.03], 'Load P3 (kN)');
128
129 % Create the dynamic labels that show current slider values
130 P1_value_label = createLabel(figure2, [0.42 0.12 0.07 0.04], sprintf('%.0f kN', P1_max));
131 P3_value_label = createLabel(figure2, [0.69 0.12 0.07 0.04], sprintf('%.0f kN', P3_max));
132
133 % Create the interactive sliders to adjust P1 and P3
134 P1slider = makeSlider(figure2, [0.24 0.12 0.17 0.04], 0, P1_max, P1_max);
135 P3slider = makeSlider(figure2, [0.51 0.12 0.17 0.04], 0, P3_max, P3_max);
136
137 % Attach the callback function so the truss updates when sliders move
138 redotruss = @(~,~) updateFigure(truss_axes, P1slider, P3slider, P1_value_label, P3_value_label, X, Y,
      Member_links);
139 set([P1slider P3slider], 'Callback', redotruss);
140
141 % Initial plot to show the truss at full load
142 updateFigure(truss_axes, P1slider, P3slider, P1_value_label, P3_value_label, X, Y, Member_links);
143
144 function updateFigure(ax, P1_sl, P3_sl, label_P1, label_P3, X, Y, M)
145     % Get current load values from the sliders
146     P1_current = P1_sl.Value;
147     P3_current = P3_sl.Value;
148
149     % Update the screen labels to match the current slider numbers
150     label_P1.String = sprintf('%.1f kN', P1_current);
151     label_P3.String = sprintf('%.1f kN', P3_current);
152
153     % Solve the truss equations for these specific loads
154     x_current = solveproblem2(P1_current, P3_current);
155     forces = x_current(4:14);
156
157     % Reset the current axes to draw the new state
158     cla(ax);
159     hold(ax, 'on');
160     title(ax, sprintf('Member Forces (P1=%.1f kN, P3=%.1f kN)', P1_current, P3_current), 'FontSize', 12);
161
162     % Loop through each member to draw the lines and add force labels
163     for i = 1:11
164         n1 = M(i,1);
165         n2 = M(i,2);
166         x_points = [X(n1), X(n2)];
167         y_points = [Y(n1), Y(n2)];
168         force = forces(i);
169
170         % Colour members based on force type: Blue for tension, Red for compression
171         if force > 1e-4
172             colour = 'b';
173         elseif force < -1e-4
174             colour = 'r';
175         else
176             colour = 'k'; % Black for zero-force members
177         end
178
179         % Draw the member and place the force value in the center
180         plot(ax, x_points, y_points, colour, 'LineWidth', 3);
181         text(ax, mean(x_points), mean(y_points) + 0.2, sprintf('%.1f kN', force),'Color', colour, '
      HorizontalAlignment', 'center', 'FontWeight', 'bold','BackgroundColor', 'white', 'EdgeColor', 'none')
      ;
182     end
183
184     % Plot the joint locations as black circles
185     scatter(ax, X, Y, 80, 'k', 'filled');
186     hold(ax, 'off');
187 end
188
189     % Helper function to create a slider control
190 function s = makeSlider(parent, position, lo, hi, val)
191     s = uicontrol('Parent',parent,'Style','slider','Units','normalized', 'Position',position,'Min',lo,'
      Max',hi,'Value',val,'SliderStep',[0.01 0.1]);
192 end
193
194     % Helper function to create a text label
195 function L = createLabel(parent, position, txt)
196     L = uicontrol('Parent',parent,'Style','text','Units','normalized', 'Position',position,'String',txt,'
      FontSize',10, 'BackgroundColor','white','HorizontalAlignment','left');
197 end
```

Problem 3 — Calibrated Structural Model

Part A: Figures and Results

Regression Calibration

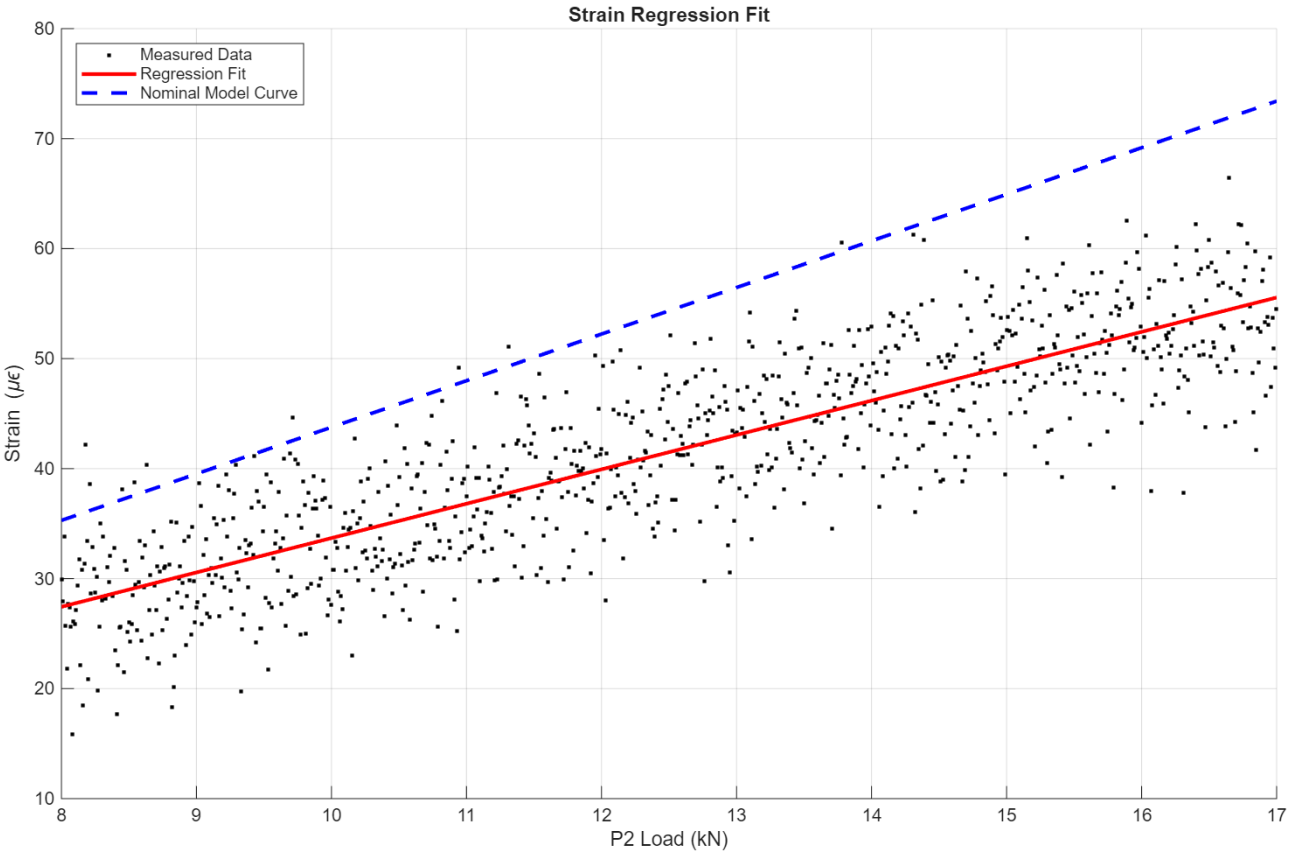


Figure 3: Measured strain data vs. P_2 load. Linear regression (red) calibrates the cross-sectional area against the nominal model (blue dashed, $A_{\text{nom}} = 660\,\text{mm}^2$).

The fitted strain–load line has equation $\varepsilon\,[\mu\varepsilon] = c\,P_2 + d$, where c and d are returned by `polyfit` [3, 7]. The calibrated area is calculated using:

$$A_{\text{cal}} = \frac{F_{\text{model}} \times 10^6}{200 \cdot (cP_2 + d)}\,[\text{mm}^2], \quad E = 200\,\text{GPa}$$

```
Calibrated Area:  894.64 mm^2 (Nominal:  660 mm^2)
Regression Coefficients:  c = 3.1243, d = 2.4434
```

Secant Root-Finding

Starting from initial points $P_2 = 17$ and $18\,\text{kN}$, just above the measured data range, the secant method [3] extrapolates to the critical load at which the stress in member F8 reaches $\sigma_{\text{allow}} = 120\,\text{MPa}$:

$$\sigma(P_2) = \frac{F_8(P_2)}{A_{\text{cal}}} = 0.120\,\text{kN/mm}^2$$

Convergence occurs in two iterations because $\sigma(P_2)$ is linear in P_2, F8 scales linearly with applied load, so the secant update is exact after one step.

Part B: Command Line Output

Iteration Table

Iter	P2 (kN)	F8 (kN)	Stress (MPa)	Error (MPa)
1	18.0000	10.2486	11.4557	-108.5443
2	191.7115	107.3563	120.0000	0.0000

Results:

Regression coefficients: $c = 3.1243$, $d = 2.4434$ (microstrain = $c \cdot P2 + d$)
Nominal Area: 660.00 mm^2
Calibrated Area: 894.64 mm^2
Critical Load $P2^*$: 191.71 kN

Part C: Engineering Interpretation

Two things drive the mismatch: random noise on the strain gauges, seen as scatter around the regression line in Figure 3, and a real offset between the as-built diagonal area and the nominal 660 mm^2 specification, picked up by the regression slope. Area-based calibration gives $A_{\text{cal}} = 894.6\,\text{mm}^2$, much larger than nominal, so the diagonal is stiffer and stronger than the design assumed, calibration therefore increases the predicted load capacity. The final secant root is $P_2^* = 191.71\,\text{kN}$, and applying a typical engineering FoS of 2 gives a recommended service limit of $P_2 \lesssim 96\,\text{kN}$.

MATLAB Code

Note: the helper function below is named `solveproblem4` *because it accepts all three loads P1, P2, and P3, and is reused in Problem 4.*

Listing 4: solveproblem4.m — General Three-Load Truss Solver Function

```
function x = solveproblem4(P1, P2, P3)
    % Function accepting P1, P2 and P3, outputting all forces.

    alpha = atan(3/1.5);
    c = cos(alpha);
    s = sin(alpha);

    % A = Matrix of unknown variables
    A = [1 0 0 1 0 0 0 0 c 0 0 0 0 0;
         0 1 0 0 0 0 0 0 s 0 0 0 0 0;
         0 0 0 -1 1 0 0 0 0 -c c 0 0 0;
         0 0 0 0 0 0 0 0 0 s s 0 0 0;
         0 0 0 0 -1 1 0 0 0 0 0 -c c 0;
         0 0 0 0 0 0 0 0 0 0 0 s s 0;
         0 0 0 0 0 -1 0 0 0 0 0 0 0 -c;
         0 0 1 0 0 0 0 0 0 0 0 0 0 s;
         0 0 0 0 0 0 1 0 -c c 0 0 0 0;
         0 0 0 0 0 0 0 0 -s -s 0 0 0 0;
         0 0 0 0 0 0 -1 1 0 0 -c c 0 0;
         0 0 0 0 0 0 0 0 0 0 -s -s 0 0;
         0 0 0 0 0 0 0 0 -1 0 0 0 0 -c c;
         0 0 0 0 0 0 0 0 0 0 0 0 -s -s];

    % b = Matrix of known variables
    b = [0;0;0;0;0;0;0;0;0;P1;0;P2;0;P3];

    % Solve for the forces
    x = A\b;
end
```

Listing 5: Problem3.m — Regression Calibration and Secant Root-Finding

```
clear; clc; close all;

% Load Data
load('synthetic_truss_295536.mat');
P1 = 13; % Fixed loads based on candidate number
P3 = 14;
A_nominal = double(areas.A_diag_mm2); % Nominal area = 660 mm^2

% Area-Based Calibration (Regression)
% Fit a straight line to the noisy strain data: eps = c*P2 + d (microstrain)
p = polyfit(P2_kN, epsilon_measured_microstrain, 1);
eps_fit = polyval(p, P2_kN);
slope = p(1); % This is the slope (microstrain/kN)
intercept = p(2); % This is the y-intercept (microstrain)

% Fit F_model vs P2 to get the force slope (dF/dP2)
p_force = polyfit(P2_kN, F_model_kN, 1);
F_slope = p_force(1); % kN of F8 per kN of P2

% Calibrated area from the slope ratio (Hooke's: dF/dP2 = E*A*deps/dP2)
```

```
% A = (dF/dP2) / (E * deps/dP2)
A_calibrated = (F_slope * 1e6) / (200 * slope);

% Nominal model strain using design area, just for the plot
eps_model = (F_model_kN * 1e6) ./ (200 * A_nominal);

% Plot Regression
figure;
hold on;
grid on;
scatter(P2_kN, epsilon_measured_microstrain, '.k', 'DisplayName', 'Measured Data');
% Smooth lines via linspace so the fit and model curves don't look jagged
P2_plot = linspace(min(P2_kN), max(P2_kN), 100);
eps_fit_plot   = polyval(p, P2_plot);
eps_model_plot = (polyval(p_force, P2_plot) * 1e6) ./ (200 * A_nominal);
plot(P2_plot, eps_fit_plot,   'r',  'LineWidth', 2, 'DisplayName', 'Regression Fit');
plot(P2_plot, eps_model_plot, 'b--','LineWidth', 2, 'DisplayName', 'Nominal Model Curve');
xlabel('P2 Load (kN)'); ylabel('Strain (\mu\epsilon)');
title('Strain Regression Fit');
legend('Location', 'northwest');

% Secant Root-Finding
fprintf('Iteration Table\n');
fprintf('%-5s | %-10s | %-12s | %-13s | %-12s\n','Iter', 'P2 (kN)', 'F8 (kN)', 'Stress (MPa)', 'Error (
    MPa)');
fprintf('%s\n', repmat('-', 1, 64));

target_stress = 0.120;   % 120 MPa in kN/mm^2
tolerance = 1e-6;
max_iteration = 50;

% Two starting points just above the data range
PreviousP = max(P2_kN);     % k-1 point [kN]
CurrentP = max(P2_kN) + 1; % k   point [kN]

for i = 1:max_iteration
    % Stress at PreviousP
    % x returns 14 elements (3 reactions + 11 members), F8 is index 3+8=11
    previous_x = solveproblem4(P1, PreviousP, P3);
    previous_stress = abs(previous_x(11)) / A_calibrated;
    previous_f = previous_stress - target_stress;

    % Stress at P_curr
    current_x = solveproblem4(P1, CurrentP, P3);
    current_force = abs(current_x(11));
    current_stress = current_force / A_calibrated;
    current_f = current_stress - target_stress;

    % Print in MPa (1 kN/mm^2 = 1000 MPa)
    fprintf('%-5d | %-10.4f | %-12.4f | %-13.4f | %-12.4f\n',i, CurrentP, current_force, current_stress
    *1000, current_f*1000);

    % Convergence check
    if abs(current_f) < tolerance
        break;
    end

    if (current_f - previous_f) == 0
        error('Secant method failed: zero slope at this iteration %d.', i);
    end

    % Secant update
    nextP = CurrentP - current_f * (CurrentP - PreviousP) / (current_f - previous_f);

    % Shift for next iteration
    PreviousP = CurrentP;
    CurrentP = nextP;
end

% Results
fprintf('\nResults:\n');
fprintf('Regression coefficients: c = %.4f, d = %.4f (microstrain = c*P2 + d)\n', slope, intercept);
fprintf('Nominal Area:    %.2f mm^2\n', A_nominal);
fprintf('Calibrated Area: %.2f mm^2\n', A_calibrated);
fprintf('Critical Load P2*: %.2f kN\n', CurrentP);
```

Problem 4 — Strain Energy in the Warren Truss

Part A: Results

Loads: $P_1 = 13$, $P_2 = 11$, $P_3 = 14\,\text{kN}$. Material: $E = 200 \times 10^6\,\text{kN/m}^2$. Member-specific areas are taken from the synthetic truss data file: $A_{\text{bot}} = 900\,\text{mm}^2$ for the bottom chord (1, 2, 3), $A_{\text{top}} = 730\,\text{mm}^2$ for the top chord (4, 5), and $A_{\text{diag}} = 660\,\text{mm}^2$ for the diagonals (6–11). Member lengths: chords $L_c = 3\,\text{m}$, diagonals $L_d = \sqrt{1.5^2 + 3^2} \approx 3.354\,\text{m}$.

The integrand for each member is the constant:

$$g_i = \frac{F_i^2}{2EA_i} \tag{2}$$

Applying the trapezoidal rule [3, 4] with $n = 50$ intervals:

$$U_i = \frac{\Delta x}{2}\left[g_i(x_0) + 2\sum_{k=1}^{n-1} g_i(x_k) + g_i(x_n)\right], \qquad U_{\text{total}} = \sum_{i=1}^{11} U_i$$

The displacement at N6 follows from Castigliano's Second Theorem [2] via central finite difference:

$$\delta = \frac{\partial U}{\partial P_2} \approx \frac{U(P_2 + \Delta P) - U(P_2 - \Delta P)}{2\,\Delta P}, \quad \Delta P = 0.1\,\text{kN}$$

and the equivalent vertical stiffness is:

$$k_{eq} = \frac{2U}{\delta^2} \tag{3}$$

Part B: Command Line Output

```
Strain Energy Results:
  Member   U_exact_kNm   U_numerical_kNm   Error_Percent
 ________ _____________ _________________ ______________

    "1"     0.00072593        0.00072593     1.4935e-14
    "2"       0.001875          0.001875     1.1565e-14
    "3"      0.0007787         0.0007787      4.177e-14
    "4"      0.0015208         0.0015208     1.4258e-14
    "5"      0.0015628         0.0015628     1.3875e-14
    "6"      0.0055337         0.0055337     1.5674e-14
    "7"     0.00050996        0.00050996      2.126e-14
    "8"     0.00050996        0.00050996            0
    "9"     0.00045173        0.00045173     1.2001e-14
   "10"     0.00045173        0.00045173     2.4001e-14
   "11"       0.005936          0.005936     1.4612e-14
 "TOTAL"      0.019856          0.019856             0


Numerical Strain Energy:  0.019856 kN-m
Exact Strain Energy:      0.019856 kN-m

Displacement at N6:       1.4712 mm
Equivalent Stiffness:     18346.68 kN/m
```

Part C: Engineering Interpretation

The numerical and exact strain energies match because $g_i(x)$ is constant along each member, and the trapezoidal rule integrates constant functions exactly for any n. The error is effectively zero, which confirms the code works correctly rather than testing integration accuracy. Each member uses its own cross-sectional area from the calibration data, so the chord and diagonal contributions to the total are weighted correctly. The displacement at N6 is $\delta \approx 1.47\,\text{mm}$, which is sensible for a 9 m span steel truss under $\sim$38 kN total loading and well within typical serviceability limits (span/360 = 25 mm).

MATLAB Code

Listing 6: Problem4.m — Trapezoidal Strain Energy and Castigliano Stiffness

```
clc; clear;

% Loads based on candidate number (kN)
P1 = 13;
P2 = 11;
P3 = 14;
E = 200e6; % Young's Modulus (kN/m^2)
n = 50; % Number of integration points

% Member-specific areas from the data file (m^2)
A_bot  = 900e-6;   % Bottom chord (1, 2, 3)
A_top  = 730e-6;   % Top chord    (4, 5)
A_diag = 660e-6;   % Diagonals    (6-11)
A_per_member = [A_bot; A_bot; A_bot; A_top; A_top; A_diag; A_diag; A_diag; A_diag; A_diag; A_diag];

% Solve for Forces
x = solveproblem4(P1, P2, P3);
F = x(4:14); % Extract just the 11 member forces

% Define Member Lengths (Members 1-5 are chords, 6-11 are diagonals)
L_chord = 3;
L_diagonal  = sqrt(1.5^2 + 3^2);
L = [ones(5,1)*L_chord; ones(6,1)*L_diagonal];

% Strain Energy Calculation
U_exact_total = 0;
U_numerical_total = 0;

% Initialize empty arrays to store the energy for each of the 11 members
U_exact_list = zeros(11, 1);
U_numerical_list = zeros(11, 1);

for i = 1:11
    % Exact calculation
    U_exact = (F(i)^2 * L(i)) / (2 * E * A_per_member(i));

    % Store the exact value
    U_exact_list(i) = U_exact;
    U_exact_total = U_exact_total + U_exact;

    % Numerical integration (Trapezoidal Rule)
    g = (F(i)^2) / (2 * E * A_per_member(i));
    dx = L(i) / n;                  % Step size

    % Create an array for the n+1 points
    g_array = g * ones(1, n+1);

    % Trapezoidal formula: dx/2 * (first + 2*middle_sum + last)
    U_numerical = (dx / 2) * (g_array(1) + 2*sum(g_array(2:n)) + g_array(n+1));

    % Store the numerical value
    U_numerical_list(i) = U_numerical;
    U_numerical_total = U_numerical_total + U_numerical;
end

% Results Table for each member
% Calculate individual errors and store in an array
Error_Percent = zeros(11, 1);
for i = 1:11
    if U_exact_list(i) == 0
        Error_Percent(i) = 0;
    else
        Error_Percent(i) = abs((U_numerical_list(i) - U_exact_list(i)) / U_exact_list(i)) * 100;
    end
end

% Calculate total error
if U_exact_total == 0
    total_error = 0;
else
    total_error = abs((U_numerical_total - U_exact_total) / U_exact_total) * 100;
end

% Assemble data arrays
Member          = [string(1:11)'; "TOTAL"];
U_exact_kNm     = [U_exact_list(:); U_exact_total];
U_numerical_kNm = [U_numerical_list(:); U_numerical_total];
Error_Percent   = [Error_Percent(:); total_error];

% Create and display the table
StrainEnergyTable = table(Member, U_exact_kNm, U_numerical_kNm, Error_Percent, 'VariableNames', {'Member'
    , 'U_exact_kNm', 'U_numerical_kNm', 'Error_Percent'});

disp('Strain Energy Results:');
disp(StrainEnergyTable);
fprintf('\n');

```

```
% Results
fprintf('Numerical Strain Energy: %.6f kN-m\n', U_numerical_total);
fprintf('Exact Strain Energy:     %.6f kN-m\n\n', U_exact_total);

% Equivalent Stiffness
dP = 0.1; % Step size for the load

% Calculate energy for P2 + dP
x_plus = solveproblem4(P1, P2 + dP, P3);
F_plus = x_plus(4:14);
U_plus = sum((F_plus.^2 .* L) ./ (2 * E * A_per_member));

% Calculate energy for P2 - dP
x_minus = solveproblem4(P1, P2 - dP, P3);
F_minus = x_minus(4:14);
U_minus = sum((F_minus.^2 .* L) ./ (2 * E * A_per_member));

% Finite difference derivative
delta = (U_plus - U_minus) / (2 * dP);

% Equivalent stiffness
k_eq = (2 * U_exact_total) / (delta^2);

% Results
fprintf('Displacement at N6:      %.4f mm\n', delta * 1000);
fprintf('Equivalent Stiffness:    %.2f kN/m\n', k_eq);
```

Problem 5 — Dynamic Response Simulator

Part A: First-Order ODE System (with hand calculations)

$$m\ddot{y}(t) + c\dot{y}(t) + ky(t) = F(t) \quad ①$$

$$x_1 = y(t)$$
$$x_2 = \dot{y}(t)$$
$$\dot{x}_1 = \dot{y}(t) = x_2$$
$$\dot{x}_2 = \ddot{y}(t)$$

Sub into ①

$$m\dot{x}_2 + cx_2 + kx_1 = F(t)$$

$$\dot{x}_2 = \frac{1}{m}\left(F(t) - cx_2 - kx_1\right)$$

$$\dot{x}_1 = x_2$$
$$\dot{x}_2 = \frac{1}{m}\left(F(t) - cx_2 - kx_1\right)$$

$$x_1(0) = y(0) = 0$$
$$x_2(0) = \dot{y}(0) = 0$$

$$\frac{d}{dt}\begin{vmatrix} x_1 \\ x_2 \end{vmatrix} = \begin{vmatrix} 0 & 1 \\ -k/m & -c/m \end{vmatrix} \begin{vmatrix} x_1 \\ x_2 \end{vmatrix} + \begin{vmatrix} 0 \\ F(t)/m \end{vmatrix}$$

The governing equation $m\ddot{y} + c\dot{y} + ky = P_0 \sin(2\pi f t)$ is rewritten as the state-space system [5], solved numerically using `ode45` [6]:

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = \frac{P_0 \sin(2\pi f t) - c\,x_2 - k\,x_1}{m}, \qquad x_1(0) = x_2(0) = 0$$

Parameters for candidate 295536 ($d_4 = 5$, $d_5 = 3$, $d_6 = 6$):

$m_o = 1,200$ kg, $k_o = 4 \times 10^6$ N/m, $\zeta_o = 0.05$, $p_0 = 10,000$ N

Parameter	Formula	Value
m	$m_o(1 + 0.02(d_4 - 5))$	1200 kg
k	$k_o(1 + 0.02(d_5 - 5))$	3.84×10^6 N/m
ζ	$\zeta_o + 0.05(d_6 - 5)$	0.10
c	$2\zeta\sqrt{km}$	13,576.45 N.s/m
f_n	$\frac{1}{2\pi}\sqrt{k/m}$	9.0032 Hz
f		6 Hz

Part B: Command Line Output & Results

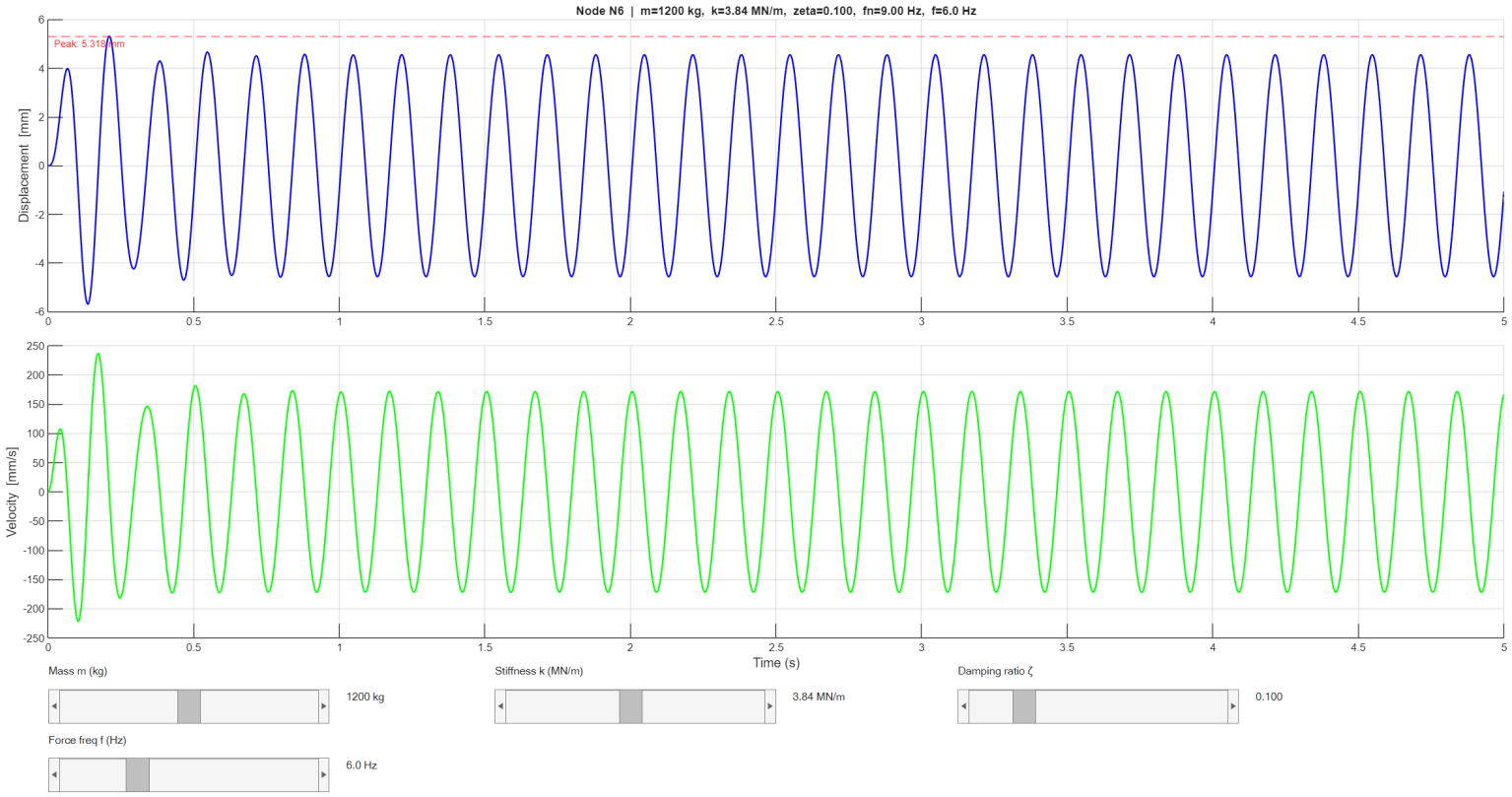


Figure 4: Dynamic displacement response at node N6 over 5 s. The interactive slider panel allows real-time parameter variation (mass 0–2400 kg, stiffness 0–8 MN/m, damping ratio 0.01–0.5, frequency 0.5–20 Hz).

Command Line Output:

```
Mass = 1200.00 kg

Stiffness = 3.8400e+06 N/m

Damping ratio = 0.1000

Damping coeff = 13576.45 N.s/m

Natural freq = 56.5685 rad/s

Natural freq = 9.0032 Hz

Forcing freq = 6.0 Hz

Static deflection = 0.002604 m
```

Time (s)	Displacement (m)
0.0	0.0000000
0.2	0.0048650
1.0	−0.0010559
3.0	−0.0010621
5.0	−0.0010623

```
Results:

Peak displacement:  5.3179 mm at t = 0.2099 s

Steady-state amplitude:  4.5557 mm

Static deflection P0/k:  2.6042 mm

Dynamic Amplif.  Factor:  1.7494

Forcing / Natural freq:  0.6664
```

Part C: Engineering Interpretation

The forcing frequency ($f = 6\,\text{Hz}$) lies below the natural frequency ($f_n \approx 9\,\text{Hz}$), so the system is in the sub-resonant regime. The Dynamic Amplification Factor of 1.75 shows the response is amplified relative to static loading, with a steady-state amplitude (4.56 mm) roughly 1.75× the static deflection (2.60 mm). This is well below the resonance peak of $1/(2\zeta) = 5$ that would occur at $f = f_n$. The transient curve decays with time constant $\tau = 1/(\zeta\omega_n) \approx 0.18\,\text{s}$, so about 5 time constants [5] ($\sim$0.9 s) are needed to reach steady state, which matches Figure 4 where transients have stopped by $t \approx 1\,\text{s}$. The interactive tool shows that sliding the frequency slider towards f_n amplifies the response sharply, illustrating resonance.

MATLAB Code

Listing 7: Problem5.m — ode45 Dynamic Solver with Interactive Parameter Sliders

```
clc; clear; close all;

% Candidate number digits
d4 = 5;
d5 = 3;
d6 = 6;

% Base values for the system
m0          = 1200; % Base mass in kg
k0          = 4e6;  % Base stiffness in N/m
damping0    = 0.05; % Base damping ratio

% Calculating parameters
mass              = m0  * (1 + 0.02*(d4 - 5));
k                 = k0 * (1 + 0.02*(d5 - 5));
damping_ratio     = damping0  + 0.05*(d6 - 5);

% Calculate damping coefficient (c = 2*zeta*sqrt(k*m))
damping_coefficient = 2 * damping_ratio * sqrt(k * mass);

% Defining the applied dynamic force F(t) = P0*sin(2*pi*f*t)
P0 = 10000; % Force amplitude in N
f  = 6;     % Forcing frequency in Hz

% Calculate natural frequencies to compare against forcing frequency
nat_freq_rad = sqrt(k/mass);           % Natural frequency in rad/s
nat_freq_hz  = nat_freq_rad / (2*pi);   % Natural frequency in Hz

fprintf('  Mass              = %.2f kg\n',     mass);
fprintf('  Stiffness         = %.4e N/m\n',    k);
fprintf('  Damping ratio     = %.4f\n',        damping_ratio);
fprintf('  Damping coeff     = %.2f N.s/m\n',  damping_coefficient);
fprintf('  Natural freq      = %.4f rad/s\n',  nat_freq_rad);
fprintf('  Natural freq      = %.4f Hz\n',     nat_freq_hz);
fprintf('  Forcing freq      = %.1f Hz\n',     f);
fprintf('  Static deflection = %.6f m\n\n',   P0/k);

% Converting the second-order ODE into a system of first-order ODEs for ode45
% q(1) = displacement (y), q(2) = velocity (y_dot)
eom = @(t, q) [q(2);
               (P0*sin(2*pi*f*t) - damping_coefficient*q(2) - k*q(1)) / mass];

% Solve the differential equation over a 5-second duration
% Initial conditions are [0; 0] (zero initial displacement and velocity)
[time_vec, state] = ode45(eom, [0 5], [0; 0], odeset('RelTol',1e-6,'AbsTol',1e-9));

% Extracting the displacement and velocity from the state matrix
displ  = state(:, 1);
veloc  = state(:, 2);

% Standardising the output table to 51 evenly spaced time steps
t_sample = linspace(0, 5, 51)';
y_sample = interp1(time_vec, displ, t_sample);

% Creating a table to display the numerical results
results_table = table(t_sample, y_sample,'VariableNames', {'Time_s', 'Displacement_m'});
disp(results_table);

[peak_displ, peak_idx]  = max(displ);
% Assuming steady state is reached in the final second of the 5s simulation
steady_displ            = max(abs(displ(time_vec >= 4)));
static_displ            = P0 / k;
dyn_amp_factor          = steady_displ / static_displ;

% Results
fprintf('\n  Results:\n');
fprintf('  Peak displacement:       %.4f mm  at t = %.4f s\n', peak_displ*1000, time_vec(peak_idx));
fprintf('  Steady-state amplitude:  %.4f mm\n',  steady_displ*1000);
fprintf('  Static deflection P0/k:  %.4f mm\n',  static_displ*1000);
fprintf('  Dynamic Amplif. Factor:  %.4f\n',     dyn_amp_factor);
fprintf('  Forcing / Natural freq:  %.4f\n',     f/nat_freq_hz);

% Create the figure window
figure = figure('Name','Interactive Dynamic Response','Color','white','Position',[80 80 1100 640]);
```

```
75
76 plot_axes = axes('Parent',figure,'Position',[0.07 0.58 0.88 0.34]);
77 grid(plot_axes,'on');
78 grid(plot_axes,'minor');
79
80 plot_axes_vel = axes('Parent',figure,'Position',[0.07 0.20 0.88 0.34]);
81 grid(plot_axes_vel,'on');
82 grid(plot_axes_vel,'minor');
83
84 % Create text labels for the parameter sliders
85 createLabel(figure, [0.07 0.14 0.13 0.03], 'Mass m (kg)');
86 createLabel(figure, [0.34 0.14 0.15 0.03], 'Stiffness k (MN/m)');
87 createLabel(figure, [0.62 0.14 0.13 0.03], 'Damping ratio ζ');
88 createLabel(figure, [0.07 0.06 0.13 0.03], 'Force freq f (Hz)');
89
90 % Create the dynamic labels that show current slider values
91 label_mass_value  = createLabel(figure, [0.25 0.10 0.07 0.04], sprintf('%.0f kg',   mass));
92 label_stiff_value = createLabel(figure, [0.52 0.10 0.07 0.04], sprintf('%.2f MN/m', k/1e6));
93 label_damp_value  = createLabel(figure, [0.80 0.10 0.07 0.04], sprintf('%.3f',      damping_ratio));
94 label_freq_value  = createLabel(figure, [0.25 0.02 0.07 0.04], sprintf('%.1f Hz',   f));
95
96 % Create the interactive sliders to adjust the parameters
97 sliderMass  = createSlider(figure, [0.07 0.10 0.17 0.04], 600,  2400, mass);
98 sliderStiff = createSlider(figure, [0.34 0.10 0.17 0.04], 1e6,  8e6,  k);
99 sliderDamp  = createSlider(figure, [0.62 0.10 0.17 0.04], 0.01, 0.5,  damping_ratio);
100 sliderFreq  = createSlider(figure, [0.07 0.02 0.17 0.04], 0.5,  20,   f);
101
102 % Attach the callback function so the plot updates when sliders move
103 redo_plot = @(~,~) solveAndPlot(plot_axes, plot_axes_vel, sliderMass, sliderStiff, sliderDamp, sliderFreq
      , label_mass_value, label_stiff_value, label_damp_value, label_freq_value, P0);
104 set([sliderMass sliderStiff sliderDamp sliderFreq], 'Callback', redo_plot);
105
106 % Initial plot call to show the response on startup
107 solveAndPlot(plot_axes, plot_axes_vel, sliderMass, sliderStiff, sliderDamp, sliderFreq, label_mass_value,
       label_stiff_value, label_damp_value, label_freq_value, P0);
108
109 function solveAndPlot(ax, ax_vel, m_sl, k_sl, z_sl, f_sl, m_label, k_label, z_label, f_label, F0)
110     % Get current parameter values from the sliders
111     mass_current = m_sl.Value;
112     k_current = k_sl.Value;
113     z_current = z_sl.Value;
114     f_current = f_sl.Value;
115
116     % Recalculate dependent variables
117     c_current = 2 * z_current * sqrt(k_current * mass_current);
118     w_current = 2 * pi * f_current;
119
120     % Update the screen labels to match the current slider numbers
121     m_label.String = sprintf('%.0f kg',   mass_current);
122     k_label.String = sprintf('%.2f MN/m', k_current/1e6);
123     z_label.String = sprintf('%.3f',      z_current);
124     f_label.String = sprintf('%.1f Hz',   f_current);
125
126     % Solve the ODE for these parameters
127     eom_current = @(t, q) [q(2); (F0*sin(w_current*t) - c_current*q(2) - k_current*q(1)) / mass_current];
128     [time_val, state_val] = ode45(eom_current, [0 5], [0; 0], odeset('RelTol',1e-6,'AbsTol',1e-9));
129
130     % Convert displacement to mm for plotting
131     y_value = state_val(:,1) * 1000;
132     v_value = state_val(:,2) * 1000;
133
134     % Find peak displacement and natural frequency for the title
135     [peak_val, peak_idx] = max(y_value);
136     fn_current = sqrt(k_current/mass_current) / (2*pi);
137
138     % Reset the current axes to draw the new state
139     cla(ax);
140     hold(ax, 'on');
141
142     % Plot the new displacement-time
143     plot(ax, time_val, y_value, 'b-', 'LineWidth', 1.5);
144
145     % Mark the peak displacement on the graph
146     yline(ax, peak_val, '--r', sprintf('Peak: %.3f mm', peak_val), 'LabelHorizontalAlignment','left', '
       LabelVerticalAlignment', 'bottom', 'FontSize',9);
147
148     % Update titles and labels
149     title(ax, sprintf('Node N6  |  m=%.0f kg,  k=%.2f MN/m, zeta=%.3f,  fn=%.2f Hz,  f=%.1f Hz',
       mass_current, k_current/1e6, z_current, fn_current, f_current), 'FontSize',11);
150     ylabel(ax,'Displacement  [mm]','FontSize',12);
151
152     grid(ax,'on');
153     grid(ax,'minor');
154     hold(ax, 'off');
155
156     cla(ax_vel);
157     hold(ax_vel, 'on');
158
159     plot(ax_vel, time_val, v_value, 'g-', 'LineWidth', 1.5);
160
161     xlabel(ax_vel,'Time (s)','FontSize',12);
```

```
    ylabel(ax_vel,'Velocity  [mm/s]','FontSize',12);

    grid(ax_vel,'on');
    grid(ax_vel,'minor');
    hold(ax_vel, 'off');

    drawnow;
end

% Helper function to create a standardised slider control
function s = createSlider(parent, position, lo, hi, val)
    s = uicontrol('Parent',parent,'Style','slider','Units','normalized', 'Position',position,'Min',lo,'
     Max',hi,'Value',val,'SliderStep',[0.01 0.1]);
end
% Helper function to create a standardised text label
function L = createLabel(parent, position, txt)
    L = uicontrol('Parent',parent,'Style','text','Units','normalized', 'Position',position,'String',txt,'
     FontSize',10, 'BackgroundColor','white','HorizontalAlignment','left');
end
```

AI Usage Disclosure

In line with the University of Sussex AI usage policy, the following AI tools were used during preparation of this portfolio. All MATLAB code, hand calculations, derivations and engineering interpretations are my own work.

I acknowledge the use of Anthropic Claude and Google Gemini, https://claude.ai/new, https://gemini.google.com/app, on a range of dates to tutor and debug. The prompts used are included below.

Problem 2 — Parametric Design Tool [18th February 2026]

1. *"How do I overlay specific contour lines (e.g. FoS = 1 and FoS = 2) on top of a* `contourf` *plot in MATLAB, and add labels to those specific contour levels?"*
2. *"How can I colour individual line segments in a MATLAB plot differently based on whether a value is positive or negative?"*

Problem 3 — Calibration and Root-Finding [4th March 2026]

1. *"What method of root finding would be most appropriate for finding the load at which stress in a truss member reaches an allowable limit, given that I need to extrapolate beyond my measured data range?"*
2. *"Can you explain what the two outputs of MATLAB's* `polyfit` *represent physically when I fit a straight line to strain vs. load data?"*

Problem 5 — Dynamic Response Simulator [21st April 2026]

1. *"How do I extract displacement values at specific time points from* `ode45` *output, given that* `ode45` *uses variable step sizes and may not land exactly on my desired sample times?"*
2. *"My* `ode45` *callback re-solves the ODE every time a slider moves, but the plot is very slow to update. Is there a way to make* `drawnow` *work more efficiently inside a slider callback?"*

References

[1] R. C. Hibbeler, *Structural Analysis*, 10th ed. Hoboken, NJ: Pearson, 2018.

[2] R. C. Hibbeler, *Mechanics of Materials*, 10th ed. Hoboken, NJ: Pearson, 2017.

[3] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed. New York, NY: McGraw-Hill, 2015.

[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge, UK: Cambridge University Press, 2007.

[5] D. J. Inman, *Engineering Vibration*, 4th ed. Hoboken, NJ: Pearson, 2014.

[6] The MathWorks, Inc., "MATLAB R2024b Documentation," 2024. [Online]. Available: `https://uk.mathworks.com/help/matlab/` [Accessed: May 2026].

[7] The MathWorks, Inc., "`polyfit` — Polynomial curve fitting," *MATLAB Documentation*, 2024. [Online]. Available: `https://uk.mathworks.com/help/matlab/ref/polyfit.html` [Accessed: May 2026].